QueryAPIGate BLOG · DESIGN NOTE

Why every front door runs the same eight steps

REST, MCP, server-sent events and the export CLI all call one engine. Identity, limits, grant, parameters, SQL guard, execution, masking, record: the order is fixed, and a step that fails stops the call with a typed error code.

Published 11 October 2026 · Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=why-every-front-door-runs-the-same-eight-steps

---

QueryAPIGate has five ways in: REST endpoints, the MCP server, server-sent events, the export CLI and the Management API. Behind all of them is one engine, and one fixed sequence.

The sequence

1. **Identify the caller.** API key, admin token, JWT or OAuth token, resolved to a named principal.
2. **Apply its limits.** Rate limit, IP allow-list, expiry.
3. **Check the grant.** This query, on this connection, in this mode.
4. **Validate parameters.** Types, ranges, lengths. Values are bound, never spliced.
5. **Guard the SQL.** Single statement, tables inside the allow-list, writes only where permitted.
6. **Run it.** A pooled connection, a time limit, a row limit, a transaction for writes.
7. **Mask and format.** Masking traced to source columns, then the format asked for.
8. **Record it.** Run history, metrics, an event; the audit log for configuration changes.

A step that fails stops the call with a typed error code, the same code whichever door the call came through.

Why it is one path

The temptation with a new transport is to bolt on what it needs and move on. The MCP server, when it arrived, could have had its own authentication and its own idea of limits. Backlog item 61 moved rate limits, metrics and authentication out of REST's Flask hooks and into the core precisely so that could not happen.

The consequence is simple to state: an AI agent cannot be governed less than an application because it arrived through a different protocol. The same grant, the same limits, the same masking rules, the same log.

What it buys you

- A security fix lands once. 0.20.1 and 0.22.1 each closed a gap in one place.
- The access map is truthful. "Who can reach what" has one answer, not one per transport.

- New front doors are cheap. Exports to object storage and incremental deliveries reused everything from step 1 to step 8.

The architecture diagram on the home page is this sequence drawn out, with the front doors on the left and the databases on the right.

READ NEXT
One worker, many threads, and what Redis is actually for

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=why-every-front-door-runs-the-same-eight-steps

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate